

MICROPROJECT REPORT

VISUALISATION OF DATA STRUCTURES

-QUEUES



Department of Computer Science and Engineering
Sree Narayana Gurukulam College of Engineering,
Kadayiruppu, 682311

SUBMITTED BY :

GROUP 2 : S3B CSE 2025-26

1)AFRAZ P M

2)AGNAL SUBY

3)AKHILA SHAJI

4)AKSHARA MOHANDAS

5)AKSHAY TANAJI METKARI

6)AKSHAYA BIJU

7)ALEENA SIBY

Under the guidance of :

MS SINDHU MP

ASSISTANT PROFESSOR

CSE DEPARTMENT

CERTIFICATE



Department of Computer Science and Engineering
Sree Narayana Gurukulam College of Engineering,
Kadayiruppu, 682311

**This is to certify that the microproject report entitled
“Queues” submitted by 8.Afraz P M ,9.Agnal Suby ,10.AkhilaShaji,
11.Akshara Mohandas,12.Akshay Tanaji Metkari,13.Akshaya
Biju & 14.Aleena Siby of S3 B.Tech Computer Science & Engineerig
during the academic year 2025-26, is a record of the work carried
out by them under our supervision.**

Faculty Guide

Asst.Prof.Sindhu M P

Head of Department

Prof(Dr)Smitha Suresh

Date:21|10|2025

TABLE OF CONTENTS

CONTENTS	PAGE NO
1.ABSTRACT	4
2.ALGORITHM	5
3.PROGRAM	8
4.OUTPUT	17
5.RESULT WITH CO	23

ABSTRACT

The objective of this micro-project is to **develop a visual representation of the Queue data structure** using C++ in Visual Studio Code with the Debugger Visualization extension. A queue is a fundamental linear data structure that follows the **First-In-First-Out (FIFO)** principle, where elements are added at the rear and removed from the front.

This project implements queue operations such as **enqueue, dequeue, peek, and display**, and provides a **pictorial visualization** of the queue during program execution. By using the Debugger Visualization tool, the elements of the queue are displayed as boxes, which update dynamically to reflect each operation, thereby creating an **animated view** of the queue. This helps users to **understand the internal working and flow of data** in the queue intuitively, making it an effective tool for learning and demonstration.

The project demonstrates how **interactive visualization** can aid in the comprehension of abstract data structures and provides a platform to **record animated representations** for reports or presentations.

ALGORITHM

Step 1: Start

- Initialize an empty queue using `deque<int> q`.
 - Declare variables `choice` and `value`.
-

Step 2: Display Menu

- Show the following options:
 - 1.Enqueue (Insert)
 - 2.Dequeue (Delete)
 - 3.Peek Front
 - 4.Display Queue
 - 5.Exit
-

Step 3: Input User Choice

- Read the user input into `choice`.
-

Step 4: Perform Operation Based on Choice

Case 1: Enqueue

- **Ask the user for the value to insert.**
- **Insert the value at the rear of the queue using `q.push_back(value)`.**
- **Update the visual representation in the Debugger Visualization panel.**

Case 2: Dequeue

- **Check if the queue is empty:**
 - **If empty, display “Queue is empty.”**
 - **If not empty, remove the front element using `q.pop_front()`.**
- **Update the visual representation.**

Case 3: Peek Front

- **Check if the queue is empty:**
 - **If empty, display “Queue is empty.”**

- If not empty, display the front element using `q.front()`.

Case 4: Display Queue

- Iterate through the queue and display all elements.

Case 5: Exit

- Terminate the program.

Default Case:

- Display “Invalid choice” and ask for input again.

Step 5: Repeat

- Go back to Step 2 until the user selects Exit.

Step 6: End

PROGRAM

C :

```
1#include <stdio.h>
```

```
2#include <stdlib.h>
```

```
3#include <stdbool.h>
```

```
4
```

```
5#define MAX_SIZE 10
```

```
6
```

```
7typedef struct {
```

```
8  int elements[MAX_SIZE];
```

```
9  int front;
```

```
10  int rear;
```

```
11  int size;
```

```
12} Queue;
```

```
13
```

```
14Queue* createQueue() {
```

```
15  Queue* q = (Queue*)malloc(sizeof(Queue));
```



```
16  q->front = 0;
17  q->rear = MAX_SIZE - 1;
18  q->size = 0;
19  return q;
20}
21
22bool isFull(Queue* q) {
23  return (q->size == MAX_SIZE);
24}
25
26bool isEmpty(Queue* q) {
27  return (q->size == 0);
28}
29
30void enqueue(Queue* q, int value) {
31  if (isFull(q)) return;
32  q->rear = (q->rear + 1) % MAX_SIZE;
33  q->elements[q->rear] = value;
```

```
34  q->size = q->size + 1;
35}
36
37int dequeue(Queue* q) {
38  if (isEmpty(q)) return -1;
39  int item = q->elements[q->front];
40  q->front = (q->front + 1) % MAX_SIZE;
41  q->size = q->size - 1;
42  return item;
43}
44
45int peek(Queue* q) {
46  if (isEmpty(q)) return -1;
47  return q->elements[q->front];
48}
```

C++ :

1#include <iostream>

2#include <vector>

3

4class Queue {

5private:

6 std::vector<int> elements;

7

8public:

9 // Add an element to the back of the queue

10 void enqueue(int value) {

11 elements.push_back(value);

12 }

13

14 // Remove an element from the front of the queue

15 int dequeue() {

16 if (elements.empty()) {

17 std::cerr << "Queue is empty!" << std::endl;

```
18     return -1; // Indicate empty queue
19 }
20 int front_element = elements.front();
21 elements.erase(elements.begin());
22 return front_element;
23 }
24
25 // Get the value of the front element
26 int peek() {
27     if (elements.empty()) {
28         std::cerr << "Queue is empty!" << std::endl;
29         return -1; // Indicate empty queue
30     }
31     return elements.front();
32 }
33
34 // Check if the queue is empty
35 bool isEmpty() {
```

```
36    return elements.empty();
37 }
38
39 // Get the number of elements in the queue
40 int size() {
41     return elements.size();
42 }
43};
```

JAVA:

```
1 import java.util.ArrayList;  
2 import java.util.NoSuchElementException;  
3  
4 class Queue {  
5     private ArrayList<Integer> elements;  
6  
7     public Queue() {  
8         elements = new ArrayList<>();  
9     }  
10  
11     // Add an element to the back of the queue  
12     void enqueue(int value) {  
13         elements.add(value);  
14     }  
15  
16     // Remove an element from the front of the queue  
17     int dequeue() {
```

```
18     if (elements.isEmpty()) {
19         throw new NoSuchElementException("Queue
is empty");
20     }
21     return elements.remove(0);
22 }
23
24 // Get the value of the front element
25 int peek() {
26     if (elements.isEmpty()) {
27         throw new NoSuchElementException("Queue
is empty");
28     }
29     return elements.get(0);
30 }
31
32 // Check if the queue is empty
33 boolean isEmpty() {
```

```
34     return elements.isEmpty();
35 }
36
37 // Get the number of elements in the queue
38 int size() {
39     return elements.size();
40 }
41 }
```


OUTPUT

Controls

Interact with the queue using the controls below.



Visualization

Front of the queue is on the left, rear is on the right.

Front

12

23

43

67

88

Rear

N

Controls

Interact with the queue using the controls below.



Enqueue



Dequeue



Peek



Visualization

Front of the queue is on the left, rear is on the right.

Front

12

23

Rear



Controls

Interact with the queue using the controls below.



Visualization

Front of the queue is on the left, rear is on the right.

Front



Rear

N

Controls

Interact with the queue using the controls below.



Visualization

Front of the queue is on the left, rear is on the right.

Front

23

43

67

88

Rear

N

Controls

Interact with the queue using the controls below.

⊕ Enqueue⊖ Dequeue👁 Peek

Visualization

Front of the queue is on the left, rear is on the right.

Queue is empty

7

Controls

Interact with the queue using the controls below.

+ Enqueue- Dequeue👁 Peek

Visualization

Front of the queue is on the left, rear is on the right.

Front

88

Rear

7

RESULT WITH CO

The queue operations were executed successfully, and the **pictorial visualization** updated dynamically in the Debugger Visualization panel & CO attained. Each element in the queue is displayed as a box, showing **real-time changes** when enqueueing or dequeueing elements.